
KDD Assessment Report

Long-Horizon Data Science Task Evaluation

Gemini 3 Pro vs. Human Expert Solution

Task Summary

Task: Anomaly Detection and Energy Loss Estimation in a DC Power Supply System

Domain: Electrical Engineering — Power Electronics / Embedded Systems

Hardware Basis: Volt3X USB-C PD Programmable Power Supply (9 V nominal, 0–600 mA, INA226 sensor, TPS55289 regulator, Raspberry Pi Pico at 1 Hz via MicroPython)

Dataset: 36,000 samples $\times$ 3 columns — 10 hours of V/I sensor data

Ground Truth: 14 injected fault events across 5 fault types

Final Answer: Total wasted energy in Joules (single verifiable scalar)

Kumbhar Digvijay Dagadu

Google Colab Environment | Python 3.10 | seed = 42

Contents

1	Introduction and Task Design	2
1.1	Motivation	2
1.2	Problem Statement	2
1.3	Why This Task is Hard for an LLM	2
2	Dataset Description	3
2.1	Generation Parameters	3
2.2	Fault Types and Physical Signatures	3
2.3	Data Quality Issues Introduced	4
3	Gemini’s Approach and Failure Analysis	4
3.1	What Gemini Did — Step by Step	4
3.2	Failure Mode 1 — Contaminated Baseline	5
3.3	Failure Mode 2 — Silent NaN Bug	5
3.4	Failure Mode 3 — Wrong Energy Formula	6
3.5	Failure Mode 4 — Current Spike Threshold Exceeds Hardware Limit	6
3.6	Failure Mode 5 — Variable Naming Inconsistency and Output Contracts	7
4	Unit Test Design and Results	7
4.1	Robust Unit Test Block Design	7
4.1.1	Design Feature 1 — Dynamic Variable Resolution	7
4.1.2	Design Feature 2 — Flexible Column Targeting	8
4.1.3	Design Feature 3 — Global Scope Pre-Flight Guard	8
4.2	Test Results — Gemini vs. Golden Solution	8
4.3	Key Observation: What the Updated Test Block Reveals	9
5	Mapping Test Results to LLM Reasoning Capabilities	10
5.1	Success: Edge-Case Handling and Boolean Logic (T6, T7)	10
5.2	Partial Success: Long-Context Tracking (T8)	10
5.3	Failure: Physical Constraint Reasoning (T1, T2)	10
5.4	Failure: Output Contract Awareness	10
6	Golden Solution Summary	11
6.1	Pipeline Overview	11
6.2	Energy Calculation — Three-Case Logic	11
6.3	Detection Thresholds and Noise Floor Justification	11
7	Conclusion	12

1. Introduction and Task Design

1.1. Motivation

This assessment required designing a *long-horizon data science task* that satisfies three conditions simultaneously:

1. The task requires reading and processing a dataset — the answer cannot be obtained by reasoning alone.
2. A human expert can solve it through careful multi-step analysis within a few hours.
3. A state-of-the-art AI agent (Gemini 3 Pro in Google Colab) is *likely to fail* due to specific, documentable reasoning weaknesses.

The domain chosen is **power electronics fault analysis**, grounded in the author’s Volt3X project — a USB-C PD programmable DC power supply built around the TPS55289 buck-boost regulator and INA226 current/voltage sensor. Every parameter in the dataset and every threshold in the solution is derived from real hardware specifications.

1.2. Problem Statement

Formal Task Statement

Given 10 hours of raw voltage and current sensor readings from a 9 V DC power supply (sampled at 1 Hz by a Raspberry Pi Pico via INA226 over I²C):

1. Load and validate the sensor data
2. Detect ALL fault/anomaly events in the time series
3. Classify each event into one of 5 fault types
4. Compute the total energy **wasted** during fault periods relative to nominal operation
5. Report the final answer: **total wasted energy in Joules**

1.3. Why This Task is Hard for an LLM

The task was deliberately engineered to expose six known LLM weaknesses:

- **NaN handling:** Python silently evaluates `NaN < threshold` as `False`, meaning unvalidated sensor dropouts pass through anomaly detection undetected.
- **Baseline contamination:** A global mean baseline is pulled toward fault values, shifting detection thresholds incorrectly.
- **Formula confusion:** Wasted energy is the *deviation* from nominal power, not the *total* power during fault windows — an order-of-magnitude difference.
- **Short circuit vs. voltage droop:** Both show low voltage; only the simultaneous high-current condition distinguishes them. LLMs often merge these.
- **Physical constraint reasoning:** Short circuit energy must be computed as $V_{\text{nom}} \times I_{\text{max}} \times \Delta t$ (maximum dissipation), not as a deviation from a near-zero actual power.

- **Event counting:** Grouping consecutive flagged samples into discrete events requires explicit block-labelling logic that LLMs frequently omit.

2. Dataset Description

2.1. Generation Parameters

The dataset was generated synthetically in `KDD_Step1_Generate_Dataset.ipynb` using `numpy.random.seed(42)` for reproducibility.

Table 1: Dataset parameters derived from Volt3X hardware specifications

Parameter	Value	Source
Nominal voltage	9.0 V	USB-PD 9V profile
Nominal current	400 mA	Mid-range test load
Current limit	600 mA	TPS55289 hardware clamp
Voltage noise σ	± 15 mV	INA226 12-bit at 9V range
Current noise σ	± 3 mA	INA226 LSB at 600mA scale
Sampling rate	1 Hz	MicroPython Pico polling loop
Total samples	36,000	10 hours $\times$ 3600 s/hr
Total fault events	14	Injected ground truth

2.2. Fault Types and Physical Signatures

Five fault types were injected with physically motivated signatures derived from Volt3X hardware behaviour:

Table 2: Ground-truth fault events injected into the dataset

Fault Type	Count	V signature	I signature	Physical cause in Volt3X
Voltage Droop	3	-0.8 to -1.5 V	slight $\uparrow$	Load surge; buck-boost recovery lag
Current Spike	4	normal	$+150$ – 200 mA	Inductive kick / capacitor charging
Short Circuit	2	collapses < 0.3 V	clamped 600 mA	Output terminals shorted; TPS55289 limits
Overvoltage	2	$+1.2$ to $+2.0$ V	normal/low	TPS55289 feedback loop instability
Sensor Dropout	3	NaN	NaN	INA226 I ² C timeout on Pico
Total	14	—	—	—

2.3. Data Quality Issues Introduced

The dataset intentionally contains the following challenges that require careful handling:

- **NaN values** at 47 sample positions (sensor dropout rows) — cannot be handled by standard comparison operators
- **Physical clipping** at $V \in [-0.5, 16]$ V and $I \in [0, 0.6]$ A (hardware limits enforced via `np.clip`)
- **Overlapping signatures** — short circuit has low voltage like a droop, but also has high current; classification must use both channels simultaneously

3. Gemini’s Approach and Failure Analysis

3.1. What Gemini Did — Step by Step

Gemini 3 Pro was given the task prompt verbatim in Google Colab AI. It generated four code cells autonomously. The following table summarises each step and compares it to the correct approach:

Table 3: Step-by-step comparison: Gemini vs. Golden Solution

Step	Gemini’s Approach	Golden Solution	Result
Data Loading	<code>pd.read_csv()</code> with no dtype specification, no NaN check, no physical bounds validation	Explicit dtypes, NaN count, physical bounds assertion	Partial
Baseline	Global <code>df['voltage_V'].mean()</code> across entire dataset including fault samples	600-sample rolling median on clean signal, edge-filled with 50th percentile	Wrong
Anomaly Detection	Global mean $\pm 2\sigma$ thresholds; attempted multi-class but used physically impossible thresholds.	Deviation from rolling baseline; 5-class fault type column; priority-ordered classification.	Partial
NaN Handling	No explicit handling — <code>NaN < threshold</code> silently evaluates to <code>False</code> ; dropouts missed	Dropout mask recorded before interpolation; dropouts explicitly labelled <code>sensor_dropout</code>	Wrong
Energy Formula	$E = \sum_{\text{fault}} P_{\text{actual}} \cdot \Delta t$ (total power, not deviation)	$E = \sum_{\text{fault}} P_{\text{actual}} - P_{\text{nominal}} \cdot \Delta t$; special case for short circuit	Wrong
Short Circuit	Merged with voltage droop (both have low V); no current condition applied	Distinct condition: $V < 0.5$ V AND $I > 550$ mA; highest priority class	Wrong

3.2. Failure Mode 1 — Contaminated Baseline

Gemini computed the baseline voltage as:

```

1 # Gemini Cell 3 --- baseline computation
2 normal_data = df[df['fault_type'] == 'normal']
3 baseline_V_normal = normal_data['voltage_V'].median()
4 P_nominal_baseline = baseline_V_normal * baseline_I_normal

```

Listing 1: Gemini’s baseline computation (incorrect)

The problem here is a **circular dependency**: `fault_type == 'normal'` is determined by Gemini’s own (incorrect) anomaly detection. Because Gemini’s detection misses sensor dropouts and some droop events, those fault samples remain labelled `normal` and contaminate the baseline median.

The golden solution avoids this by computing a rolling median on the *smoothed* signal before any classification:

```

1 # Cell 4 --- rolling median baseline, robust to outliers
2 df['V_baseline'] = (df['voltage_V_s']
3                     .rolling(window=600, center=True, min_periods
4                               =60)
5                     .median())
6 V_nominal = float(df['voltage_V_s'].quantile(0.50))
7 df['V_baseline'] = df['V_baseline'].fillna(V_nominal)

```

Listing 2: Golden solution baseline computation (correct)

A 600-second rolling median is robust because a 45-second fault event barely affects the median of a 600-sample window. Using `center=True` ensures zero phase lag.

3.3. Failure Mode 2 — Silent NaN Bug

This is the most subtle failure. In Python/NumPy:

$$\text{NaN} < 8.5 \longrightarrow \text{False} \quad (\text{silently, no exception}) \quad (1)$$

Gemini’s threshold comparisons therefore evaluate `False` for every dropout row, causing all 47 sensor dropout samples to be classified as `normal`. This is documented by unit test 7:

```

1 def test_7_sensor_dropout_not_classified_as_normal(self):
2     raw = pd.read_csv('/content/drive/MyDrive/KDD/
3                       volt3x_sensor_data.csv')
4     dropout_idx = raw[raw['voltage_V'].isnull()].index
5     leaked = (df.loc[dropout_idx, 'anomaly'] == False).sum()
6     self.assertEqual(leaked, 0,
7                      f"{leaked} NaN rows NOT flagged as anomaly --- NaN
                        comparison silently False.")

```

Listing 3: Unit test exposing Gemini’s NaN bug

The golden solution records the dropout mask *before* any interpolation or smoothing, then applies it directly:

```

1 # Cell 3 --- record dropout positions BEFORE interpolation
2 dropout_mask = df_raw['voltage_V'].isnull() # True where INA226
   returned NaN
3
4 # Later in Cell 5 --- apply directly to fault_type
5 df.loc[dropout_mask, 'fault_type'] = 'sensor_dropout'

```

Listing 4: Golden solution NaN handling (correct)

3.4. Failure Mode 3 — Wrong Energy Formula

Gemini calculated total power during fault windows, not the *deviation* from nominal:

$$E_{\text{Gemini}} = \sum_{t \in \text{fault}} V(t) \cdot I(t) \cdot \Delta t \quad (\text{WRONG}) \quad (2)$$

The correct formula is the deviation from what would have been delivered under normal operation:

$$E_{\text{wasted}} = \sum_{t \in \text{fault}} |P_{\text{actual}}(t) - P_{\text{nominal}}| \cdot \Delta t \quad (\text{CORRECT for regular faults}) \quad (3)$$

For short circuits specifically, the actual power $P_{\text{actual}} = V \cdot I \approx 0.1 \times 0.6 \approx 0.06 \text{ W}$, which would make the deviation formula report *less* wasted energy than a droop event. This is physically wrong — a short circuit is the most destructive fault. The correct treatment is:

$$E_{\text{wasted}}^{\text{short}} = V_{\text{nominal}} \times I_{\text{max}} \times \Delta t = 9.0 \times 0.6 \times \Delta t = 5.4 \text{ W} \cdot \Delta t \quad (4)$$

This is implemented in Cell 7 of the golden solution.

3.5. Failure Mode 4 — Current Spike Threshold Exceeds Hardware Limit

While Gemini successfully classified most fault types, it completely failed to detect the `current_spike` events. Gemini set the current spike detection threshold at `CURRENT_SPIKE_THRESHOLD = 0.65 A`, but the hardware current limit of the Volt3X is 0.60 A (enforced by the TPS55289 and `np.clip`). This means Gemini’s threshold can never trigger — no sample in the dataset can ever reach 0.65 A. As a result, the current spike class was entirely absent from Gemini’s output, causing unit test T2 to fail:

Unit Test 2 — Gemini FAIL (actual output)

T2 FAIL Missing fault types: {'current_spike'}

This is a physical domain reasoning failure: Gemini set a threshold without checking it against the hardware constraint it was explicitly told about in the prompt (“600mA current limit”). The correct threshold is any relative value below 600 mA, such as the golden solution’s `I_SPIKE_THRESH = +100 mA` above the dynamic baseline.

3.6. Failure Mode 5 — Variable Naming Inconsistency and Output Contracts

In its initial code generation, Gemini named its final energy variable `total_wasted_energy_joules`, but the strict unit tests originally expected `total_fault_energy`. In early iterations, this caused tests T3 and T4 to crash catastrophically with a `NameError` rather than gracefully evaluating the mathematics.

This represents a failure of **output contract awareness** — Gemini does not inherently reason about how its produced variables will be consumed by downstream code or test harnesses. A human engineer writing production code would either follow the agreed variable naming convention or explicitly document their chosen names.

To objectively grade the LLM’s mathematical reasoning rather than penalising a syntax quirk, the master unit test framework was upgraded to dynamically resolve variables using `globals().get()` and flexible column targeting. Once this safety net was applied, Gemini’s execution stabilised, allowing it to successfully pass the physical bounds tests (T3 and T4) and yield an incomplete but mathematically valid partial energy calculation of 175.82 J.

4. Unit Test Design and Results

4.1. Robust Unit Test Block Design

The master unit test framework used across all three submission notebooks was engineered specifically to handle the unpredictable output structures characteristic of Large Language Models. Because LLMs frequently deviate from strict output contracts and invent ad-hoc variable names, a traditional `unittest` framework with rigid assertions would immediately crash with a `NameError`, ruining the automated evaluation. To prevent this, the test block was proactively designed with three core resilience mechanisms to ensure the agent is graded strictly on its mathematical logic rather than penalised for arbitrary syntax quirks.

4.1.1. Design Feature 1 — Dynamic Variable Resolution

Instead of rigidly demanding a single variable name for the final energy scalar, the test block dynamically searches the global scope for a wide range of plausible names the LLM might have generated.

```

1 # Dynamically find the energy variable
2 energy_val = None
3 for var_name in ['total_fault_energy', '
4     total_wasted_energy_joules',
5     'total_energy', 'E_total', 'total_waste']:
6     if var_name in globals():
7         energy_val = globals()[var_name]
8         break

```

Listing 5: Resilient energy variable lookup

This design guarantees that the physical constraint tests evaluate the actual computed value rather than failing on a technicality. Tests T3 and T4 are subsequently merged into

a single robust assertion block to seamlessly verify both that the value is mathematically finite and that it falls within the physically realistic bounds of 50 to 500 J.

4.1.2. Design Feature 2 — Flexible Column Targeting

LLMs are highly inconsistent with DataFrame column naming conventions, often alternating between names like `anomaly` or `is_anomaly`, and `fault_type` or `fault_class`. The framework anticipates this by dynamically resolving the anomaly mask and target columns before executing the core tests:

```

1 # Dynamically find the anomaly mask to prevent KeyErrors
2 if 'is_anomaly' in df.columns:
3     is_anom = df['is_anomaly'] == True
4 elif 'anomaly' in df.columns:
5     is_anom = df['anomaly'] == True
6 elif 'fault_type' in df.columns:
7     is_anom = df['fault_type'] != 'normal'

```

Listing 6: Column name flexibility

4.1.3. Design Feature 3 — Global Scope Pre-Flight Guard

A pre-flight assertion at the top of the script catches total pipeline execution failures before any downstream tests are permitted to run. This provides a clean, easily readable execution error rather than a cascade of uninterpretable, downstream test failures:

```

1 assert 'df' in globals(), \
2     "Execution Error: DataFrame 'df' is missing from the global
   scope."

```

Listing 7: Global scope pre-flight check

4.2. Test Results — Gemini vs. Golden Solution

Eight unit tests verify correctness, output structure, physical plausibility, and constraint satisfaction. Results below reflect the actual Gemini run.

Metric	Gemini Final Answer	Golden Solution
Total Energy Wasted	175.82 J	470.59 J
Fault Event Count	10 out of 14 events detected.	14 out of 14 events detected.
Fault Types Classified	Missing <code>current_spike</code> .	All 5 fault types classified.

Table 4: Performance Comparison: Gemini vs Golden Solution

Gemini Final Score: 6/8 tests passed (actual run)**FAIL (2 tests):**

- **T1** — Expected 14 fault events, got 10 (missed 4 events).
- **T2** — `current_spike` absent from output (missing fault type).

PASS (6 tests):

- **T3/4** — Energy is valid and in realistic bounds (175.82 J).
- **T5** — Normal samples contributed zero waste.
- **T6** — Short circuit identified separately.
- **T7** — Sensor dropout not classified as normal.
- **T8** — Baseline not contaminated (8.9901 V).

Golden Solution Score: 8/8 tests passed

All tests pass. Correct event count (14), all 5 fault types classified, energy in valid range, zero waste for normal samples, short circuit distinct, NaN handled explicitly, baseline within 0.1 V of nominal (tighter than the 0.5 V bound).

4.3. Key Observation: What the Updated Test Block Reveals

With the newly updated and robust test framework, Gemini successfully passes 6 out of the 8 unit tests. The two remaining test failures accurately expose the LLM’s failure to capture specific physical edge cases in the data: **T1** (it detected only 10 of 14 fault events) and **T2** (the `current_spike` fault type was completely missing from the classification).

However, because the updated unit test block is highly flexible in how it resolves variables (e.g., dynamically locating the anomaly mask and auto-calculating the energy sum), Gemini is no longer penalised for arbitrary naming conventions. It successfully passes the physical constraint tests (T3/T4, T5, T6, T7, and T8), reporting a plausible but incomplete energy value of 175.82 J. In contrast, the Golden Solution, using domain-specific engineering logic, achieves a perfect 8/8, accurately identifying all 14 events and integrating the true total of 470.59 J of wasted energy.

5. Mapping Test Results to LLM Reasoning Capabilities

The updated evaluation (where Gemini passed 6 out of 8 tests) maps directly to fundamental reasoning capabilities of the LLM. Importantly, Gemini demonstrated strong competence in standard programmatic logic, but struggled significantly when domain-specific physics were required.

5.1. Success: Edge-Case Handling and Boolean Logic (T6, T7)

Contrary to typical LLM pitfalls, Gemini successfully isolated sensor dropouts (passing **T7**) and correctly disambiguated short circuits from standard droops (passing **T6**). This demonstrates that the LLM has strong built-in templates for handling standard Python and `pandas` edge cases (such as missing NaN data filtering) when generating dataframe manipulations.

5.2. Partial Success: Long-Context Tracking (T8)

Gemini successfully established a clean baseline of 8.9901 V, safely passing the ± 0.5 V contamination threshold (**T8**). However, it relied on a global statistical approach rather than a robust, human-engineered rolling median. While statistically sufficient to pass the grading threshold, it highlights that LLMs favour simple, single-step mathematical approximations over advanced, context-aware signal processing techniques.

5.3. Failure: Physical Constraint Reasoning (T1, T2)

The most critical failures occurred where code intersected with physical reality. By setting a `current_spike` threshold of 0.65 A on a circuit physically clamped at 0.60 A, Gemini proved it treats prompt parameters as arbitrary statistical suggestions rather than hard physical limits. This lack of physical grounding caused the complete omission of the `current_spike` fault class (**T2**) and resulted in 4 of the 14 total events going entirely undetected (**T1**). Furthermore, its formula for short circuit energy lacked the specific physics (maximum current into near-zero load) required for an accurate calculation.

5.4. Failure: Output Contract Awareness

As detailed earlier, Gemini generated the correct mathematical logic for energy integration but assigned it to an unexpected variable name (`total_wasted_energy_joules`). A human engineer working in a collaborative codebase inherently checks what variable names downstream test harnesses expect. Gemini lacks this persistent “contract awareness,” generating code that is logically correct in isolation but fragile when integrated into a wider, rigid deployment pipeline.

6. Golden Solution Summary

6.1. Pipeline Overview

Table 5: Golden solution pipeline — `golden.solution.ipynb`

Cell	Stage	Key design decision
1	Imports & paths	Fixed <code>SEED=42</code> ; all file paths defined once
2	Load & validate	Explicit dtype check; NaN count; physical bounds assertion
3	Data cleaning	Dropout mask saved first; linear interpolation (max 30 samples); <code>medfilt</code> kernel=5
4	Baseline reconstruction	600-sample rolling median; <code>center=True</code> ; edge-filled with global 50th percentile
5	Fault detection	Deviation thresholds $\gg$ noise floor; priority-ordered 5-class classification
6	Event grouping	<code>_block</code> column; consecutive samples grouped into discrete events
7	Energy calculation	Three cases: regular faults, short circuits, dropouts
8	Final answer	Scalar in Joules; Wh conversion; loss fraction
9	Visualisation	3-panel plot: voltage, current, fault classification strip
10	Unit tests	All 8 tests pass

6.2. Energy Calculation — Three-Case Logic

$$E_{\text{wasted}}(t) = \begin{cases} |P_{\text{actual}}(t) - P_{\text{nominal}}| \cdot \Delta t & \text{regular faults (droop, spike, overvoltage)} \\ V_{\text{nom}} \cdot I_{\text{max}} \cdot \Delta t & \text{short circuit (maximum dissipation)} \\ P_{\text{nominal}} \cdot \Delta t & \text{sensor dropout (conservative estimate)} \\ 0 & \text{normal operation} \end{cases} \quad (5)$$

6.3. Detection Thresholds and Noise Floor Justification

All thresholds are set at least $30\times$ above the INA226 noise floor:

Table 6: Detection thresholds vs. sensor noise floor

Fault Type	Threshold	Noise floor	SNR (ratio)
Voltage Droop	$\Delta V < -500 \text{ mV}$	$\pm 15 \text{ mV}$	$33\times$
Current Spike	$\Delta I > +100 \text{ mA}$	$\pm 3 \text{ mA}$	$33\times$
Short Circuit	$V < 500 \text{ mV (abs.)}$	$\pm 15 \text{ mV}$	$33\times$
Overvoltage	$\Delta V > +800 \text{ mV}$	$\pm 15 \text{ mV}$	$53\times$
Sensor Dropout	<code>isnull()</code>	N/A	N/A

7. Conclusion

This assessment successfully designed and executed a long-horizon data science task grounded in real-world electrical engineering constraints. The Volt3X fault detection dataset proved to be a highly effective benchmark for evaluating the limitations of current state-of-the-art Large Language Models.

While Gemini 3 Pro demonstrated strong fundamental capabilities in generating `pandas` code and applying basic statistical thresholds, it ultimately failed to solve the core engineering problem. Its approach exposed critical weaknesses in multi-step causal reasoning, domain-specific physical constraints (such as setting a current threshold above the hardware’s physical clamp limit), and the handling of silent programmatic edge cases like `NaN` propagation.

Thanks to the deployment of a highly resilient, crash-proof unit testing framework, the evaluation objectively measured these shortcomings without catastrophic execution failures. The results are definitive: the human expert pipeline accurately reconstructed the pristine baseline, handled all sensor dropouts, and classified all 14 fault events, yielding the true total wasted energy of 470.59 J. In contrast, the AI agent captured only 10 events and missed an entire fault category, resulting in a severely underestimated 175.82 J.

Ultimately, this study demonstrates that while LLMs are powerful accelerators for boilerplate code generation, they currently lack the physical intuition, global context awareness, and rigorous pipeline design required to autonomously solve complex, domain-specific engineering tasks.

Final Answer Comparison

Gemini Final Result: 175.82 J of total wasted energy calculated.

Gemini Event Count: 10 events detected out of 14 ground-truth events (missed 4 events, failed **T1**).

Gemini Classification: Failed to classify `current_spike` events (**T2 FAIL**).

Golden Solution: 470.59 J of total wasted energy accurately calculated; successfully classified all 14 events across all 5 fault types.

Unit Tests Passed: Gemini **6/8** | Golden solution **8/8**

Test Block Note: The master unit test framework successfully evaluated both solutions using robust dynamic variable resolution, preventing catastrophic crashes and exposing only the true mathematical shortcomings of the LLM.